Design and Analysis of Algorithms Project Report

Arnav Kanaujia
Souparna Pal

October 27, 2026

1 Problem

Lectures in an educational institution run around the clock. There are n lectures and the schedule of the lectures is the same every day. The duration of lectures may vary from course to course. There are some lectures which spans across different days, for example, starting at 11 PM and ends at 1 AM the next day. Hence the earliest lecture or last lecture is not well defined.

The supervisor of the institution wants to check on every lecture in a day. For this, she makes a series of visits such that at the time of a visit, she checks on every lecture happening at that instant. In a day, there are m discrete times when the supervisor is free for visits. Also, in order to minimally disturb the lectures, it is advised that every lecture is checked exactly once. Give an algorithm that outputs the minimum number of visits the supervisor has to make so as to check on every lecture happening in a day exactly once, if that is possible.

1.1 Assumptions

The implementation uses the following assumptions.

1. Time values are input in the format `HH:MM`.
2. Lecture intervals include their start and end times.
3. A lecture with ending time less than starting time crosses midnight.
4. Duplicate available visit times are treated as one distinct visit time.
5. A lecture interval with equal start and end times is interpreted as being of a single instant. A full-day lecture can't be represented.

2 Algorithm Description

2.1 Input

Lecture times are input in the form of their start and end times in the format `HH:MM`. Visiting times are input in the format `HH:MM`. Thus the input is a list of length n containing tuples and a list of length m containing strings.

2.2 Convert to minutes after midnight

The lecture start and end times and the visiting times are converted to minutes after midnight. This is done using the function `ToMinutes`. After doing this, we obtain

$$L = \{[s_i, e_i] : 1 \leq i \leq n\}, \qquad T = \{t_i : 1 \leq i \leq m\},$$

where L contains the start and end times of each lecture in minutes after midnight and T contains the visiting times in minutes after midnight.

The problem asks us to select a subset of T such that each interval in L contains exactly one element of T and the size of the subset is minimized.

2.3 Linearize the problem

The elements of L and T can be thought of as lying on a circle, since lectures can cross midnight. In this step, we try to convert the minimization problem on the circle to a minimization problem on a line. First the list T is sorted in ascending order of minutes after midnight. So it can be assumed that $t_i < t_{i+1}$ for all $1 \leq i \leq m-1$.

A visiting time $t_i \in T$ is chosen. This is selected as a mandatory visit and we try to minimize the number of visits required to cover the lectures, not already covered by t_i, exactly once. Then we can minimize over each choice of t_i to obtain the final answer. The list T is rotated to obtain a list q with $q[0] = t_i$, i.e. $q = \{t_i, \ldots, t_m, t_1, \ldots t_{i-1}\}$.

Convention: We write $t \in [s, e]$ if time t lies in the lecture $[s, e]$, i.e. one of the following holds-

- $s \leq e$ and $s \leq t \leq e$,
- $s > e$ and $s \leq t$,
- $s > e$ and $t \leq e$.

The last two cases are when the lecture is crossing midnight. The function `Contains` is used to determine if $t \in [s, e]$.

If some visiting times are inside a lecture interval already covered by t_i, then those visiting times can't be selected again, since a lecture can't be visited twice. Thus we define the set F_i of all forbidden visiting times for choice of $t_i \in T$.

$$F_i = \{t_j \in T : t_j \in [s_k, e_k],\ \ t_i \in [s_k, e_k]\}$$

We convert each $[s, e] \in L$ to an interval in terms of the minimum and maximum index of visiting times in q contained in it. $[s_i, e_i]$ is identified with (l_i, r_i), where $l_i = min\{i : q[i] \in [s_i, e_i]\}$ and $r_i = max\{i : q[i] \in [s_i, e_i]\}$. If $q[0] \in [s_i, e_i]$ then $[s_i, e_i]$ is identified with ϕ as this interval is already covered by t_i and thus we don't want to consider it in the linear problem. If $\{i : q[i] \in [s_i, e_i]\}$ is empty, then $[s_i, e_i]$ is identified with ∞. This is a convention, which helps us identify when there is an interval that contains no visiting time. This is helpful as in such situations there is no viable solution. These identifications are done using the function `LectureInterval`.

Define

$$I = \{(l_i, r_i) : [s_i, e_i] \text{ is identified with } (l_i, r_i),\ \ 1 \leq i \leq n\}.$$

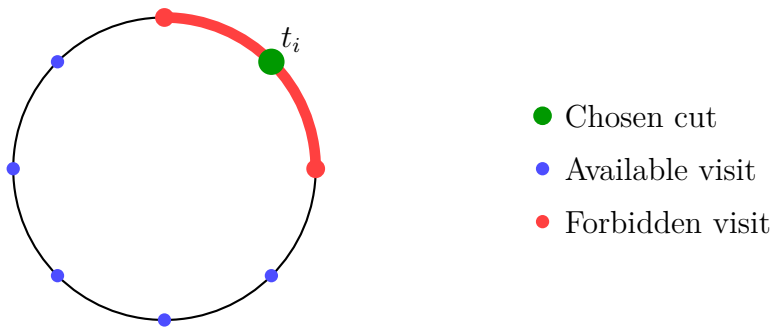


Figure 1: Choosing a mandatory visit makes other visit times inside lectures containing it forbidden.

If $\infty \in I$, then there is no viable solution. Also if for some $1 \leq j \leq n$, $\{q[l_j], q[l_j + 1], \ldots, q[r_j]\} \subset F_i$, i.e. the interval is only covered by forbidden visiting times, then too there is no viable solution. If there is no viable solution, we take the solution to be ∞, by convention. All the above steps are done by the function `PreprocessCut`.

By choosing a $t_i \in T$ as a mandatory visit, we are basically cutting the circle at t_i to get a linear problem of covering intervals not already covered by t_i using minimum number of non-forbidden visit times. These reduced sets of intervals and visit times are contiguous and we use a dynamic programming argument to solve the problem for these reduced sets. We get such a linear problem for each choice of cut.

2.4 Dynamic Programming in linear problem

We are given a set of contiguous intervals (those not covered by $q[0]$) and a set of visit times (non-forbidden ones). We need to find the minimum number of visit times required to cover each interval exactly once.

$$\begin{aligned} \mathrm{OPT}[a][b] = &\text{ minimum number of visits required to cover exactly once} \\ &\text{all lecture intervals covered by visits } q[i] \text{ with indices } i \leq b, \qquad 0 \leq a < b \leq m. \\ &\text{subject to the last two visits occurring at indices } a \text{ and } b, \end{aligned}$$

We say that a sequences of indices in q ending in b is **valid** if visiting times with indices from this sequence cover all intervals that can be covered by visit times with indices up to b and the indices do not correspond to forbidden visit times. Thus we minimise over the length of all valid sequences ending in b. Define $\mathrm{OPT}[a][b] = \infty$, if no such sequence exists. We want to define $\mathrm{OPT}[b][c]$ for $c > b$ when a valid sequence ending in b exists.

We claim that there exists a valid sequence ending in b and c, given a valid sequence ending in a and b if and only if there exists no interval $(l, r) \in I$ such that $l > b$ and $r < c$ or $l \leq b$ and $r \geq c$.

For each $c \in \{1, \ldots, m\}$, the function `IsValid` maintains a pair

$$\mathrm{valid}[c-1] = (L_c, R_c).$$

Initially, the bounds are set to $L_c = 0$ and $R_c = c$. The function then processes every interval $(l, r) \in I$ as follows:

- If $c \leq r$, the right bound is updated to $R_c \leftarrow \min(R_c, l)$. This update is performed for every $c \in \{1, \ldots, r\}$.

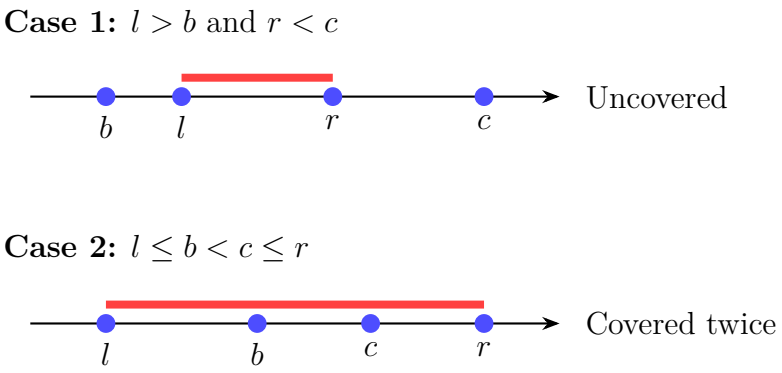


Figure 2: The two cases in which a transition from b to c invalid.

- If $c > r$, the left bound is updated to $L_c \leftarrow \max(L_c, l)$. This update is performed for every $c \in \{r+1, \ldots, m\}$.

We claim that there exists a valid sequence ending in b and c, given a valid sequence ending in a and b if and only if

$$L_c \leq b < R_c.$$

We prove both of these claims later.

Now q has indices $0, \ldots, m-1$. Also $l, r \neq 0$, since intervals covered by $q[0]$ have been removed in previous step. Introduce m such that $q[m] > q[p]$ for all $0 \leq p \leq m-1$. The index m will act as a visit time in infinity.

Base case:

$$\mathrm{OPT}[0][a] = \begin{cases} 1, & \text{if } \nexists\, (l, r) \text{ with } r < a \quad (0 < l \text{ is always true}), \\ \infty, & \text{o.w.} \end{cases}$$

By our definition, $\mathrm{OPT}[0][a]$ is defined if there exists a valid sequence ending in 0 and a. The valid sequence exists if and only if there exists no interval $(l, r) \in I$ such that $l > 0$ and $r < a$ or $l \leq 0$ and $r \geq a$, by a previous claim. Now, $l \leq 0$ is not possible as intervals containing $q[0]$ have been removed and $l > 0$ is always true. So $\mathrm{OPT}[0][a]$ is defined if there exists no interval $(l, r) \in I$ such that $r < a$. In this case, we define the number to be 1 as only index a is required to cover all suitable intervals (0 is a mandatory visit and is added later).

Recurrence:

$$\mathrm{OPT}[b][c] = \min_{0 \leq a \leq b-1} \Big(\mathrm{OPT}[a][b]\Big) + K, \quad \text{if } \exists \text{ a valid sequence ending in } b, c \text{ and } c \notin F_i,$$

where

$$K = \begin{cases} 0, & c = m, \\ 1, & c \neq m. \end{cases}$$

By our definition, $\mathrm{OPT}[b][c]$ is defined if there exists a valid sequence ending in b and c. Thus $\mathrm{OPT}[b][c]$ will be the minimum over length of valid sequences ending in a, b and c, $0 \leq a \leq b-1$. Length of such sequences is the length up to a, b plus 1 if $c \neq m$, i.e. if c is an actual visit index. But if $c = m$ then we add nothing as m is an imaginary index

added for our convenience and will not be counted in the solution. Now if we minimise the length of valid sequences up to a, b, we get OPT$[a][b]$. Hence, OPT$[b][c]$ turns out to be minimum of OPT$[a][b]$ added with 1 or 0 as needed.

Answer:

$$\text{Answer} = \min_{0 \le b \le m-1} \text{OPT}[b][m]$$

For the answer, we need minimum length of valid sequences covering all intervals, i.e. ending in index m, since all intervals are covered by indices up to m. Thus, we take minimum over all possible b of minimum length of valid sequences ending in b and m, which is just the minimum over OPT$[b][m]$.

The DP is done in the function SolveLineDP. The proof of correctness of the DP is done later.

2.5 Combine to get solution

If $n = 0$, answer is 0 as there are no intervals to cover. Else if $m = 0$, then an answer does not exist as we can cover no intervals with zero visit times. Let $m, n \neq 0$.

$$\text{Final answer} = \left(\min_{\text{whenever valid}} \text{Line DP for each cut} \right) + 1.$$

We claim that the final solution is the minimum of the solutions of the linear problems obtained by each choice of cut plus 1 to account for the mandatory visit $q[0]$. This claim is proven later. This minimisation over all cuts is done using the function `MinVisits`.

3 Pseudocode

Algorithm 1 ToMinutes

Require: Time string t in `HH:MM` format
Ensure: Total minutes elapsed since midnight
1: Split t at ":" into hours h and minutes m
2: Convert h and m into integers
3: **return** $60h + m$

Algorithm 2 Contains

Require: Start time s, end time e, time t
Ensure: TRUE if t belongs to the interval $[s, e]$, FALSE otherwise
1: **if** $s \le e$ **then**
2: **return** $(s \le t \wedge t \le e)$
3: **else**
4: **return** $(t \ge s \vee t \le e)$
5: **end if**

Algorithm 3 LectureInterval

Require: Lecture interval (s, e), rotated visit list q
Ensure: NONE if $q[0]$ satisfies the lecture; INFINITY if no visit time can satisfy it; otherwise (l, r)

if Contains$(s, e, q[0])$ **then**
    return NONE
end if
indices $\leftarrow$ empty list
for $i = 0$ **to** length$(q) - 1$ **do**
    if Contains$(s, e, q[i])$ **then**
        Append i to *indices*
    end if
end for
if *indices* is empty **then**
    return INFINITY
end if
$l \leftarrow \min(indices)$
$r \leftarrow \max(indices)$
return (l, r)

Algorithm 4 IsValid

Require: Number of visit times m, intervals (l, r)
Ensure: Array of allowable predecessor bounds

for $c = 0$ **to** $m - 1$ **do**
    $valid[c] \leftarrow (0, c + 1)$
end for
for each lecture interval (l, r) in $intervals$ **do**
    for $c = 1$ **to** r **do**
        $valid[c - 1].upper \leftarrow \min(valid[c - 1].upper, l)$
    end for
    for $c = r + 1$ **to** m **do**
        $valid[c - 1].lower \leftarrow \max(valid[c - 1].lower, l)$
    end for
end for
return $valid$

Algorithm 5 PreprocessCut

Require: Sorted visit times $times$, lecture intervals $lectures$, index of the mandatory first visit cut_index

Ensure: $(q, intervals, forbidden)$ if valid; NONE otherwise

$q \leftarrow times[cut_index : end]$
$q \leftarrow q$ concatenated with $times[0 : cut_index]$
$forbidden \leftarrow$ empty set
for each lecture (s, e) in $lectures$ **do**
    if Contains$(s, e, q[0])$ **then**
        for $i = 1$ **to** length$(q) - 1$ **do**
            if Contains$(s, e, q[i])$ **then**
                Add i to $forbidden$
            end if
        end for
    end if
end for
$intervals \leftarrow$ empty list
for each lecture (s, e) in $lectures$ **do**
    $interval \leftarrow$ LectureInterval(s, e, q)
    if $interval = NONE$ **then**
        continue
    end if
    if $interval = INFINITY$ **then**
        return NONE
    end if
    $(l, r) \leftarrow interval$
    $all_forbidden \leftarrow TRUE$
    for $i = l$ **to** r **do**
        if $i \notin forbidden$ **then**
            $all_forbidden \leftarrow FALSE$
            break
        end if
    end for
    if $all_forbidden = TRUE$ **then**
        return NONE
    end if
    Append (l, r) to $intervals$
end for
return $(q, intervals, forbidden)$

Algorithm 6 SolveLineDP

Require: Number of visit times m, intervals, forbidden indices
Ensure: Minimum number of additional visits, or INFINITY if infeasible

$min_right \leftarrow \min\{r : (l, r) \in intervals\}$
$valid \leftarrow \text{IsValid}(m, intervals)$
for $a = 0$ **to** $m - 1$ **do**
    for $b = 1$ **to** m **do**
        $OPT[a][b] \leftarrow INFINITY$
    end for
end for
for $b = 1$ **to** $m - 1$ **do**
    if $b \notin forbidden$ **and** $b \leq min_right$ **then**
        $OPT[0][b] \leftarrow 1$
    end if
end for
for $c = 1$ **to** m **do**
    if $c < m$ **and** $c \in forbidden$ **then**
        continue
    end if
    for $b = 1$ **to** $c - 1$ **do**
        if $b < valid[c-1].lower$ **or** $b \geq valid[c-1].upper$ **then**
            continue
        end if
        $min_prev \leftarrow INFINITY$
        for $a = 0$ **to** $b - 1$ **do**
            $min_prev \leftarrow \min(min_prev, OPT[a][b])$
        end for
        if $min_prev \neq INFINITY$ **then**
            if $c = m$ **then**
                $cost_increment \leftarrow 0$
            else
                $cost_increment \leftarrow 1$
            end if
            $OPT[b][c] \leftarrow \min(OPT[b][c], min_prev + cost_increment)$
        end if
    end for
end for
$answer \leftarrow \min\{OPT[b][m] : 1 \leq b \leq m - 1\}$
return $answer$

4 Time Complexity

Given

n = number of lectures, m = number of distinct available visit times.

The complexity below is measured in terms of these two parameters. The input list `times_input` has m entries.

Algorithm 7 MinVisits

Require: Visit times *times_input*, lecture intervals *lectures_input*
Ensure: Minimum number of visits required, or INFINITY if impossible

times ← empty list
for each time t in *times_input* **do**
    Append ToMinutes(t) to *times*
end for
Sort *times* in ascending order and remove duplicates
lectures ← empty list
for each lecture (s, e) in *lectures_input* **do**
    Append (ToMinutes(s), ToMinutes(e)) to *lectures*
end for
if *lectures* is empty **then**
    return 0
end if
$m \leftarrow$ length(*times*)
if $m = 0$ **then**
    return INFINITY
end if
$answer \leftarrow INFINITY$
for $cut_index = 0$ **to** $m - 1$ **do**
    $result \leftarrow$ PreprocessCut($times, lectures, cut_index$)
    if $result = NONE$ **then**
        continue
    end if
    $(q, intervals, forbidden) \leftarrow result$
    if *intervals* is empty **then**
        $answer \leftarrow \min(answer, 1)$
        continue
    end if
    $dp_answer \leftarrow$ SolveLineDP($m, intervals, forbidden$)
    if $dp_answer \neq INFINITY$ **then**
        $answer \leftarrow \min(answer, 1 + dp_answer)$
    end if
end for
return *answer*

Built-ins such as `min`, `max`, `all` and list comprehensions take time linear in the length of the iterable they scan.

4.1 `MinVisits`

`MinVisits` calls `PreprocessCut` to rotate the times and compute the lecture intervals and the forbidden set, and then `SolveLineDP` to solve the resulting problem on a line. Both of these calls occur exactly m times. The cost is therefore

$$T(n, m) \;=\; T_{\text{setup}} \;+\; m \cdot \Big(T_{\text{preprocess}} + T_{\text{dp}}\Big).$$

Converting the m visit times and the $2n$ lecture endpoints with `ToMinutes` costs $\mathcal{O}(m+n)$. Building the set, sorting it and getting rid of duplicates costs $\mathcal{O}(m \log m)$. Hence

$$T_{\text{setup}} = \mathcal{O}(m \log m + n).$$

4.2 `Contains`

It has a constant number of comparisons, so $\mathcal{O}(1)$.

4.3 `LectureIntervals`

It has one call to `Contains`, then a list comprehension that calls `Contains` for each of the m entries of `q`, then `min` and `max` over at most m indices, so $\mathcal{O}(m)$.

4.4 `PreprocessCut`

The function does four things, and we add up their costs one after another.

First, it builds the rotated list `q = times[cut_index:] + times[:cut_index]`. Slicing and concatenating copy at most m elements, so this step costs $\mathcal{O}(m)$.

Second, it builds the set `forbidden`. For each of the n lectures it performs one $\mathcal{O}(1)$ test of whether the lecture contains `q[0]`. If it does, the function loops over the remaining $m-1$ entries of `q` and calls `contains` on each of them, which costs $\mathcal{O}(m)$. In the worst case every lecture contains `q[0]`, so this step costs $n \cdot \mathcal{O}(m) = \mathcal{O}(nm)$.

Third, for each lecture it calls `lecture_interval`, which costs $\mathcal{O}(m)$ as shown in the previous section. Over all n lectures this is $n \cdot \mathcal{O}(m) = \mathcal{O}(nm)$.

Fourth, for each lecture with interval (l, r) it evaluates `all(i in forbidden for i in range(l, r+1))`. This examines at most $r - l + 1 \le m$ indices, each with an $\mathcal{O}(1)$ set lookup, so it costs $\mathcal{O}(m)$ per lecture and $n \cdot \mathcal{O}(m) = \mathcal{O}(nm)$ over all lectures.

Adding the four steps gives

$$T_{\text{preprocess}} = \mathcal{O}(m) + \mathcal{O}(nm) + \mathcal{O}(nm) + \mathcal{O}(nm) = \mathcal{O}(nm).$$

4.5 `SolveLineDP`

This function is called only when the list of intervals is non-empty, and it receives at most n intervals. As before, we add up the costs of its parts in the order in which they are executed.

Computing `min_right` is a single pass over the intervals and costs $\mathcal{O}(n)$. Allocating `OPT`, an $m \times (m+1)$ table filled with `INF`, costs $\mathcal{O}(m^2)$. Building `valid` with `IsValid`

starts with a list of m entries, which costs $\mathcal{O}(m)$. Then, for every interval (l, r), two loops run over $c = 1, \ldots, r$ and $c = r + 1, \ldots, m$, and together they perform exactly m constant-time updates. With at most n intervals this is $\mathcal{O}(nm)$, so `is_valid` costs $\mathcal{O}(m + nm) = \mathcal{O}(nm)$ in total. Finally, initialising row 0 of `OPT` costs $\mathcal{O}(m)$.

The main part of the function is the pair of nested loops

```
for c in range(1, m + 1):
    ...
    for b in range(1, c):
        ...
        min_prev = min(OPT[a][b] for a in range(b))
```

The statement that computes `min_prev` scans b table entries, so one iteration of the inner loop costs $\mathcal{O}(b)$, plus $\mathcal{O}(1)$ for the checks and the assignment. The two `continue` conditions can skip iterations, but in the worst case none of them fires, so the total work is bounded by

$$\sum_{c=1}^{m} \sum_{b=1}^{c-1} \Big(b + \mathcal{O}(1)\Big) \;=\; \sum_{c=1}^{m} \left(\frac{c(c-1)}{2} + \mathcal{O}(c)\right) \;=\; \frac{(m-1)\,m\,(m+1)}{6} + \mathcal{O}(m^2) \;=\; \mathcal{O}(m^3).$$

After the loops, the answer is extracted with `min(OPT[b][m] for b in range(m))`, which scans m entries and costs $\mathcal{O}(m)$.

Adding the preparation, the main loops, and the final extraction gives

$$T_{\text{dp}} = \mathcal{O}(n) + \mathcal{O}(m^2) + \mathcal{O}(nm) + \mathcal{O}(m) + \mathcal{O}(m^3) + \mathcal{O}(m) = \mathcal{O}(m^3 + nm).$$

4.6 Total running time

The cost of one cut is

$$T_{\text{preprocess}} + T_{\text{dp}} = \mathcal{O}(nm) + \mathcal{O}(m^3 + nm) = \mathcal{O}(m^3 + nm).$$

Multiplying by the m cuts and adding the setup cost gives

$$T(n, m) = \mathcal{O}(m \log m + n) + m \cdot \mathcal{O}(m^3 + nm) = \mathcal{O}\Big(m \log m + n + m^4 + n\,m^2\Big).$$

Since $m \log m = \mathcal{O}(m^4)$ and $n = \mathcal{O}(n\,m^2)$, the setup terms are absorbed, and

$$\boxed{T(n, m) = \mathcal{O}\Big(m^4 + n\,m^2\Big)}$$

5 Proof of Correctness

5.1 Existence of valid sequence

Lemma 1. *Suppose a valid sequence ending in a, b exists, where $a < b < c$. Then it can be extended to a valid sequence ending in b, c if and only if there is no interval $(l, r) \in I$ satisfying either*

$$l > b \text{ and } r < c,$$

or

$$l \le b \text{ and } r \ge c.$$

Proof. Suppose a valid extension ending in b, c exists. Consider any interval $(l, r) \in I$.

If $l > b$ and $r < c$, then the interval lies strictly between b and c. Since b and c are consecutive indices, the interval is not covered, contradicting validity.

If $l \leq b$ and $r \geq c$, then

$$l \leq b < c \leq r.$$

Thus, both b and c belong to the interval. Since both are selected, the interval is covered at least twice, again contradicting validity.

Therefore, neither configuration can occur.

Suppose no interval satisfies either of the two configurations. We show that appending c to a valid sequence ending in a, b preserves validity.

Consider any interval $(l, r) \in I$. There are three cases.

- **Case 1:** $r < b$**.** The interval lies entirely before b. Its coverage is already determined by the existing valid sequence, so appending c does not affect it.
- **Case 2:** $l \leq b \leq r$**.** The interval contains b, so it is already covered by the existing sequence. Since the configuration $l \leq b$ and $r \geq c$ is excluded, we must have $r < c$. Consequently, $c \notin [l, r]$, and the interval remains covered exactly once.
- **Case 3:** $l > b$**.** Since the configuration $l > b$ and $r < c$ is excluded, we must have $r \geq c$. Hence,

  $$l > b \quad \text{and} \quad r \geq c,$$

  so $c \in [l, r]$. None of the earlier selected indices belongs to this interval, since they are all at most $b < l$. Therefore, the interval is covered exactly once by c.

In every case, the interval remains covered exactly once. Furthermore, c is assumed not to be a forbidden index. Therefore, appending c produces a valid sequence. □

Lemma 2. *Let*

$$L_c = \max\left(\{l : (l, r) \in I,\ r < c\} \cup \{0\}\right)$$

and

$$R_c = \min\left(\{l : (l, r) \in I,\ r \geq c\} \cup \{c\}\right).$$

Then a valid extension from a sequence ending in a, b to one ending in b, c exists if and only if

$$L_c \leq b < R_c.$$

Proof. By Lemma 1, the extension is valid if and only if neither of the two forbidden configurations occurs.

First, consider the configuration

$$l > b \quad \text{and} \quad r < c.$$

To exclude this configuration, every interval satisfying $r < c$ must also satisfy $l \leq b$. Equivalently,

$$b \geq \max\left(\{l : (l, r) \in I,\ r < c\} \cup \{0\}\right) = L_c.$$

Next, consider the configuration

$$l \leq b \quad \text{and} \quad r \geq c.$$

To exclude this configuration, every interval satisfying $r \geq c$ must also satisfy $l > b$. Equivalently,

$$b < \min\left(\{l : (l, r) \in I,\ r \geq c\} \cup \{c\}\right) = R_c.$$

Combining these inequalities, we obtain

$$L_c \leq b < R_c.$$

Conversely, suppose $L_c \leq b < R_c$. By the definition of L_c, every interval with $r < c$ satisfies $l \leq b$. Thus, no interval can satisfy $l > b$ and $r < c$. Similarly, by the definition of R_c, every interval with $r \geq c$ satisfies $l > b$. Thus, no interval can satisfy $l \leq b$ and $r \geq c$.

Neither forbidden configuration can occur. By Claim 1, the extension is therefore valid. Hence, proved. □

5.2 Dynamic Programming in linear problem

We consider the linearized problem obtained after fixing the mandatory visit at index 0. The actual visit indices are $0, 1, \ldots, m-1$, and we introduce an index m at infinity, which represents the end of the sequence and does not correspond to an actual visit.

Recall that I is the set of intervals (l, r) that remain to be covered, and that F is the set of forbidden visit indices. For $1 \leq c \leq m$, let L_c and R_c be the bounds defined by the function `IsValid`.

By the transition criterion established previously, a valid sequence ending in a, b can be extended to one ending in b, c if and only if

$$L_c \leq b < R_c.$$

For an actual visit index $c < m$, we must additionally require $c \notin F$.

For $0 \leq a < b \leq m$, define $\mathrm{OPT}[a][b]$ to be the minimum number of actual visits, excluding the mandatory visit at index 0, among all valid sequences ending in a, b and covering every interval that must be covered by indices up to b. The index m is allowed as a terminal index but is not counted as a visit. If no such sequence exists, we define $\mathrm{OPT}[a][b] = \infty$.

Theorem 1 (Optimal substructure)**.** *Suppose a valid sequence ending in a, b, c is optimal among all valid sequences ending in b, c. Then its prefix ending in a, b is optimal for the state* $\mathrm{OPT}[a][b]$.

Proof. Suppose the given sequence is

$$S = (0, \ldots, a, b, c),$$

and let C be the number of actual visits in its prefix ending in a, b. Assume, for a contradiction, that this prefix is not optimal for the state $\mathrm{OPT}[a][b]$.

Then there exists a valid sequence

$$S' = (0, \ldots, a, b)$$

with the same last two indices a, b and with fewer visits than the original prefix.

Consider the sequence obtained by appending c to S'. Since the transition from b to c is valid, this extension is valid as well. This is because the validity of the transition depends on the pair b, c and the interval constraints, not on the earlier part of the sequence. The resulting sequence ends in b, c and has fewer visits than S, contradicting the assumed optimality of S.

Therefore, the prefix ending in a, b is optimal for $\mathrm{OPT}[a][b]$. □

Since index 0 is the mandatory visit, the initial state is

$$\mathrm{OPT}[0][b] = \begin{cases} 1, & \text{if } b \notin F \text{ and } r \geq b \text{ for every } (l, r) \in I, \\ \infty, & \text{otherwise,} \end{cases} \qquad 1 \leq b < m.$$

The condition on the right endpoints ensures that no interval has already been passed without being covered. The visit at b is counted, whereas the mandatory visit at 0 is excluded from the DP cost and will be added separately. Equivalently, the base case is finite precisely when b is not forbidden and there is no interval $(l, r) \in I$ with $r < b$.

Suppose that all states $\mathrm{OPT}[a][b]$ with $b < c$ have been computed correctly. We wish to compute $\mathrm{OPT}[b][c]$, where $0 \leq b < c \leq m$. For $c < m$, the index c must not be forbidden. Moreover, the transition from b to c is valid precisely when

$$L_c \leq b < R_c.$$

Define the transition cost

$$K(c) = \begin{cases} 1, & c < m, \\ 0, & c = m. \end{cases}$$

The recurrence is

$$\boxed{\mathrm{OPT}[b][c] = \begin{cases} \min\limits_{0 \leq a < b} \mathrm{OPT}[a][b] + K(c), & L_c \leq b < R_c, \\ & c = m \text{ or } c \notin F, \\ \infty, & \text{otherwise.} \end{cases}}$$

Proof of the recurrence. Consider any valid sequence ending in b, c. Let a be the index immediately preceding b. Its prefix ending in a, b must be valid. By the optimal-substructure theorem, this prefix can be replaced by an optimal sequence for the state $\mathrm{OPT}[a][b]$ without affecting the validity of the transition from b to c.

Conversely, take an optimal sequence represented by $\mathrm{OPT}[a][b]$. If $L_c \leq b < R_c$ and either $c = m$ or $c \notin F$, appending c produces a valid sequence ending in b, c. The additional cost is 1 when c is an actual visit and 0 when c is the terminal index at infinity.

Every valid sequence ending in b, c has some predecessor $a < b$, so minimizing over all such predecessors considers every possible valid sequence. If the transition conditions fail, no valid extension from b to c exists, and the corresponding state is ∞. Therefore, the recurrence is correct. □

The terminal index m lies beyond every interval in I. Consequently, a valid sequence ending in b, m represents a sequence in which every remaining interval has been covered exactly once. Since m is not an actual visit, the DP cost does not include it. The minimum number of visits required for the linearized problem, excluding the mandatory visit at index 0, is therefore

$$\min_{0 \leq b < m} \mathrm{OPT}[b][m].$$

5.3 Combining to get solution

Theorem 2. *The minimum number of visits required to cover every lecture interval on the circle exactly once is equal to the minimum, over all choices of mandatory visit times, of the optimal solutions to the corresponding linear sub-problems, plus one for the mandatory visit.*

Proof. Let T be the set of available visit times, and let L be the set of lecture intervals on the circle. If $L = \varnothing$, no visits are required, and the result is immediate. Otherwise, suppose a feasible solution exists.

Let $S \subseteq T$ be any feasible solution to the circular problem. Since $L \neq \varnothing$, every lecture interval must contain exactly one selected visit. In particular, S is non-empty. Choose any $t_i \in S$ and use it as the mandatory visit. Rotate the visiting times so that t_i becomes index 0. Every lecture interval containing t_i is already covered by this visit. Since S is feasible, no other selected visit can lie in any such interval. Hence, all other selected visits belong to the set of non-forbidden indices. Remove the lecture intervals containing t_i. The remaining intervals must each be covered exactly once by the other visits in S. After cutting the circle at t_i and rotating the indices, these intervals and the remaining available visits form the corresponding linear sub-problem.

Thus, $S \setminus \{t_i\}$ is a feasible solution to that linear sub-problem. Consequently, the optimal value of this linear sub-problem, plus one for the mandatory visit, is at most $|S|$.

Conversely, consider any feasible solution to a linear sub-problem obtained by choosing t_i as the mandatory visit. Add t_i to the visits selected by the linear solution. Every lecture interval containing t_i is covered by t_i. The remaining lecture intervals are covered exactly once by the linear solution. Moreover, the preprocessing step ensures that none of the selected visits in the linear solution belongs to a lecture interval already covered by t_i.

Therefore, every lecture interval on the circle is covered exactly once. The resulting set of visits is a feasible solution to the circular problem. It follows that the optimal value of the circular problem is at most the optimal value of each linear sub-problem, plus one. Taking the minimum over all choices of t_i gives the reverse inequality.

Combining the two inequalities, we obtain

$$\boxed{\mathrm{OPT}_{\text{circle}} = 1 + \min_{t_i \in T} \mathrm{OPT}_{\text{linear}}(t_i).}$$

Hence, proved. □

6 Implementation

The algorithm has been implemented in python. This is the link to the implementation.

7 Conclusion

In this project, we developed a dynamic programming algorithm to solve the problem. We began by formulating the problem on a circle and then considered every possible choice of cut to transform it into a collection of linear sub-problems. Each linear sub-problem was solved using dynamic programming, and the solutions obtained for all possible cuts were combined to determine the global optimum.

The worst-case time complexity of the supplied implementation is

$$O(nm^2 + m^4).$$